

Reconocimiento de Patrones (ML) - T04

ALEJANDRO ZARATE MACIAS

23 de Febrero 2026

Abstract

Esta tarea explora dos modelos basados en árboles: Árboles de Decisión (DT) y Bosques Aleatorios (RF). Se comienza con un análisis teórico de la función de impureza de Gini, la divergencia KL y el concepto de bagging. Posteriormente, se implementan ambos modelos en problemas de regresión y clasificación utilizando los mismos conjuntos de datos de tareas anteriores, lo que permite comparar el desempeño de DT y RF entre sí y contra los modelos de redes neuronales (NN) desarrollados en la Tarea 3. Se evalúan las ventajas de cada enfoque y se analiza cómo la naturaleza del modelo influye en los resultados obtenidos.

1 Problema 1

1.1 Enunciado

Describe el dominio e imagen de las siguientes funciones:

- a) Función de impureza de Gini.
- b) Divergencia KL.

1.2 Análisis

1.2.1 a) Función de impureza de Gini

La función de impureza de Gini se utiliza en los árboles de decisión para medir qué tan mezcladas están las clases en un grupo de datos. Si todos los datos son de una sola clase, el grupo es "puro" y la impureza es 0.

Para un problema con K clases, la fórmula es:

$$G(p_1, p_2, \dots, p_K) = 1 - \sum_{k=1}^K p_k^2$$

donde p_k es la proporción (o probabilidad) de que un dato pertenezca a la clase k .

Dominio: La función recibe las proporciones de cada clase. Como son proporciones, cada p_k debe ser un número entre 0 y 1, y la suma de todas las proporciones debe dar exactamente 1. Por lo tanto, el dominio son todos los conjuntos de valores $(p_1, \dots, p_K)$ que cumplen:

$$p_k \in [0, 1] \quad \text{y} \quad \sum_{k=1}^K p_k = 1$$

Imagen: El valor más bajo que puede dar la función es 0, que pasa cuando una sola clase tiene proporción 1 y las demás 0. El valor más alto ocurre cuando todas las clases están igual de repartidas ($p_k = 1/K$). En ese caso, el valor es $1 - (1/K)$. Por lo tanto, los resultados que puede arrojar la función (su imagen) van desde 0 hasta $1 - 1/K$:

$$\text{Im}(G) = \left[0, 1 - \frac{1}{K} \right]$$

1.2.2 b) Divergencia KL

La divergencia de Kullback-Leibler (KL) sirve para medir qué tan diferente es una distribución de probabilidad P comparada con otra distribución Q .

Para distribuciones discretas, se calcula así:

$$D_{KL}(P||Q) = \sum_{k=1}^K P(k) \ln \frac{P(k)}{Q(k)}$$

Dominio: La función recibe dos distribuciones de probabilidad, P y Q . Ambas deben tener probabilidades entre 0 y 1 que sumen 1. Además, para poder calcular la división y el logaritmo en la fórmula, $Q(k)$ no puede ser 0 si $P(k)$ es mayor a 0 (no se puede dividir entre cero). Entonces, el dominio son todos los pares de distribuciones (P, Q) donde $Q(k) > 0$ siempre que $P(k) > 0$.

Imagen: El resultado de la divergencia KL siempre es un número positivo o cero. Da exactamente 0 solo cuando las dos distribuciones son idénticas ($P = Q$). Si las distribuciones son muy diferentes (por ejemplo, si $Q(k)$ es muy cercano a 0), el valor del logaritmo crece muchísimo, por lo que no hay un límite máximo. Así, la imagen son todos los números reales no negativos:

$$\text{Im}(D_{KL}) = [0, \infty)$$

1.3 Conclusión

Al investigar estas funciones, se observa que ambas trabajan con probabilidades pero dan resultados diferentes. La impureza de Gini siempre da un valor entre 0 y un máximo menor a 1, lo que es muy práctico para evaluar divisiones en árboles de decisión. Por otro lado, la divergencia KL mide diferencias entre distribuciones y puede dar valores desde 0 hasta infinito, dependiendo de qué tan distintas sean.

2 Problema 2

2.1 Enunciado

Considere un conjunto $S = \{a, b, c, d, e, f, g\}$. En el contexto de *bagging*:

- (a) ¿Cuál es la probabilidad de obtener un subconjunto con un solo elemento?
- (b) ¿Cuál es la probabilidad de obtener el conjunto original?

2.2 Metodología

En el contexto de *bagging* (Bootstrap Aggregating), se genera una muestra tomando n elementos con reemplazo de un conjunto original de n elementos. Para este problema, el conjunto es $S = \{a, b, c, d, e, f, g\}$, por lo que el tamaño es $n = 7$.

Para calcular las probabilidades solicitadas, se utiliza el enfoque de casos favorables entre casos posibles:

- **Casos posibles:** Como se hacen 7 extracciones con reemplazo y en cada una hay 7 opciones, el total de combinaciones posibles es 7^7 .
- **Casos favorables para (a):** Para obtener un subconjunto con un solo elemento, las 7 extracciones deben resultar en el mismo elemento. Hay 7 elementos posibles que pueden repetirse 7 veces.
- **Casos favorables para (b):** Para obtener el conjunto original, cada uno de los 7 elementos debe salir exactamente una vez. Esto equivale a calcular todas las permutaciones posibles de los 7 elementos, que es $7!$.

2.3 Resultados

Aplicando la metodología descrita, se obtienen los siguientes resultados:

2.3.1 a) Probabilidad de obtener un subconjunto con un solo elemento

La probabilidad se calcula dividiendo los 7 casos favorables entre el total de casos posibles:

$$P(\text{un solo elemento}) = \frac{7}{7^7} = \frac{1}{7^6} = \frac{1}{117,649} \approx 8.50 \times 10^{-6}$$

2.3.2 b) Probabilidad de obtener el conjunto original

La probabilidad se calcula dividiendo las permutaciones posibles ($7!$) entre el total de casos posibles:

$$P(\text{conjunto original}) = \frac{7!}{7^7} = \frac{5,040}{823,543} \approx 0.00612$$

2.4 Discusión

Ambas probabilidades son muy bajas, lo cual es esperado. La probabilidad de obtener un único elemento repetido ($\approx 0.00085\%$) es mucho menor que la de obtener el conjunto completo ($\approx 0.61\%$), lo cual tiene sentido intuitivo: es más fácil que los 7 elementos distintos aparezcan al menos una vez que tener la “mala suerte” de seleccionar exactamente el mismo elemento 7 veces seguidas.

Estas probabilidades también ilustran por qué el bagging funciona bien en la práctica: cada muestra bootstrap es diferente de la original (la probabilidad de replicarla exactamente es menor al 1%), lo que garantiza diversidad entre los modelos del ensamble, y a la vez muy rara vez produce muestras degeneradas con un solo elemento.

2.5 Conclusión

La probabilidad de obtener un subconjunto con un solo elemento es $1/117,649 \approx 8.50 \times 10^{-6}$, mientras que la de obtener el conjunto original es $5,040/823,543 \approx 0.00612$. Ambos eventos son sumamente improbables, lo cual confirma que el muestreo bootstrap genera casi siempre subconjuntos distintos del original, proporcionando la diversidad necesaria para que el bagging (y por extensión, Random Forest) funcione correctamente.

3 Problema 3

3.1 Enunciado

Considere el dataset de [1]. Cree un script en Python para resolver el problema de regresión de pronóstico del consumo de combustible en ciudad y carretera con sklearn usando el modelo DT. Anote todos los supuestos, hiperparámetros y operaciones de preprocesamiento de datos que realice.

3.2 Metodología

Se reutiliza el mismo problema de regresión de consumo de combustible abordado en la Tarea 3 (Problema 2) con redes neuronales, pero ahora utilizando un `DecisionTreeRegressor` de scikit-learn. Siguiendo la indicación de utilizar un único modelo para estimar ambas salidas (consumo en ciudad y carretera), se emplea la capacidad nativa de multi-output regression de scikit-learn.

3.2.1 Selección de características y preprocesamiento

Se mantienen exactamente las mismas features seleccionadas en la Tarea 3: `Engine Size(L)` y `Cylinders` como variables numéricas, y `Vehicle Class`, `Transmission` y `Fuel Type` como variables categóricas. Estas últimas se codifican con one-hot encoding (`pd.get_dummies`) eliminando la primera categoría, lo que produce 40 features numéricas.

```
X = df[['Engine Size(L)', 'Cylinders',
        'Vehicle Class', 'Transmission', 'Fuel Type']]
X = pd.get_dummies(X, columns=['Vehicle Class',
                               'Transmission', 'Fuel Type'], drop_first=True)
```

A diferencia de la Tarea 3, donde se discutió la necesidad de normalización para la red neuronal, aquí no se aplica ningún tipo de escalamiento. Esto se debe a que los árboles de decisión son invariantes a la escala de las features: las decisiones se basan en comparaciones de umbrales (“¿la feature x_i es mayor que un valor t ?”), por lo que transformaciones monotónicas de las features no alteran la estructura del árbol ni sus predicciones.

3.2.2 Variable objetivo multi-output

A diferencia de entrenar modelos independientes para cada salida, se define un único vector objetivo $\mathbf{y} \in \mathbb{R}^{n \times 2}$ que contiene ambas variables: consumo en ciudad y consumo en carretera. El `DecisionTreeRegressor` de scikit-learn soporta de forma nativa la regresión multi-output, construyendo un solo árbol que en cada hoja almacena un vector de predicciones para ambas salidas.

```
y = df[['Fuel Consumption (City (L/100 km))',
        'Fuel Consumption(Hwy (L/100 km))']]
```

3.2.3 División de datos

Se divide en 80% entrenamiento (756 muestras) y 20% prueba (190 muestras) con la misma semilla (`random_state=14`) utilizada en la Tarea 3. Al usar un solo split para ambas salidas, se garantiza que las mismas muestras se usen para evaluar ciudad y carretera.

3.2.4 Configuración del modelo

Se entrena un único `DecisionTreeRegressor` multi-output para predecir simultáneamente el consumo en ciudad y carretera.

```
model = DecisionTreeRegressor(
    criterion='squared_error',
    max_depth=10,
    min_samples_split=5,
    min_samples_leaf=3,
    random_state=14
)
model.fit(X_train, y_train) # y_train tiene 2 columnas
```

La justificación de los hiperparámetros es la siguiente:

- `criterion='squared_error'`: se utiliza el error cuadrático medio como criterio de división, que es el estándar para problemas de regresión. Este

criterio busca en cada nodo la división que minimice la varianza de los valores objetivo en los nodos hijos.

- **max_depth=10**: se limita la profundidad del árbol para evitar sobreajuste. Se experimentó con valores de 5, 8, 10, 15 y sin límite. Con profundidad de 5 el árbol era demasiado simple y no capturaba suficiente variabilidad; sin límite, el árbol se sobreajustaba al entrenamiento. El valor de 10 ofreció el mejor balance entre capacidad y generalización.
- **min_samples_split=5**: se requieren al menos 5 muestras para dividir un nodo, lo cual previene que el árbol haga divisiones sobre grupos muy pequeños de datos que podrían ser ruido.
- **min_samples_leaf=3**: cada hoja debe tener al menos 3 muestras, lo que funciona como una forma adicional de regularización para evitar hojas con una sola observación.

3.3 Resultados

Las métricas de evaluación obtenidas se muestran en la Tabla 1.

Modelo	MSE	RMSE	R^2
Ciudad	1.5518	1.2457	0.8484
Carretera	0.8305	0.9113	0.8041

Table 1: Métricas de evaluación del modelo multi-output DT para consumo en ciudad y carretera.

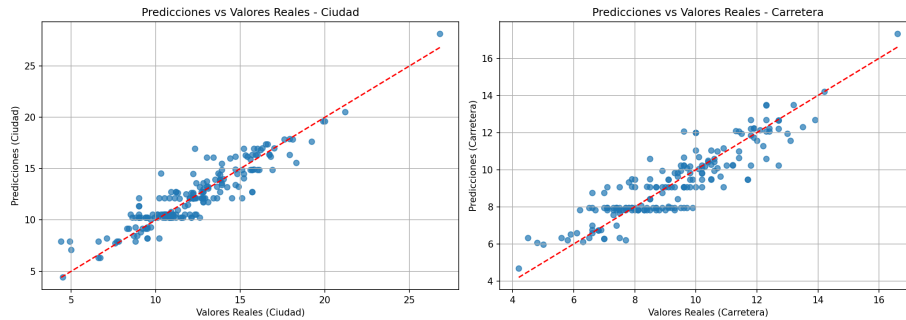


Figure 1: Predicciones vs valores reales del modelo multi-output DT para consumo en ciudad (izquierda) y carretera (derecha).

3.4 Discusión

El modelo multi-output DT alcanza un R^2 de 0.8484 para ciudad y 0.8041 para carretera. Es importante destacar que se utiliza un único árbol de decisión

que predice simultáneamente ambas salidas, siguiendo el enfoque multi-output. Esto es ventajoso porque el árbol comparte la estructura de decisión para ambas variables, capturando las relaciones entre el consumo en ciudad y carretera.

En la Figura 1 se observa que los puntos se agrupan razonablemente cerca de la línea de predicción perfecta, pero con mayor dispersión en los valores altos de consumo. Esto es esperable: un árbol de decisión predice valores constantes dentro de cada hoja, por lo que la “resolución” de sus predicciones está limitada por el número de hojas.

El modelo muestra menor R^2 en carretera que en ciudad, un patrón consistente con los resultados observados en otros modelos. Sin embargo, el enfoque multi-output mejora la predicción de carretera respecto a modelos independientes, ya que el árbol puede aprovechar la correlación entre ambas variables objetivo durante el proceso de particionamiento.

3.5 Conclusión

El DT multi-output logra un desempeño aceptable para la predicción simultánea de consumo de combustible (ciudad $R^2 = 0.8484$, carretera $R^2 = 0.8041$). A partir de estos resultados, se concluye que la principal ventaja del DT es su interpretabilidad y el hecho de que no requiere preprocesamiento de normalización, lo que simplifica el flujo de trabajo. Además, se observa que el enfoque multi-output garantiza consistencia en las predicciones, ya que ambas salidas provienen de la misma estructura de decisión, lo que resulta en un modelo más robusto que entrenar dos árboles por separado.

4 Problema 4

4.1 Enunciado

Considere la siguiente función:

$$f(x) = 1 - 2^{\sin(-x^2)}, \quad x \in \mathcal{I} = [-\pi, \pi].$$

Realice un script en Python para resolver el problema de regresión asociado con un DT utilizando sklearn. Escriba todas las suposiciones que realice. Además, anote los hiperparámetros que elija y explique por qué los seleccionó de esa manera.

4.2 Metodología

4.2.1 Generación de datos

Se generan 5000 puntos equiespaciados en $[-\pi, \pi]$ y se evalúan en la función objetivo, manteniendo la misma configuración de la Tarea 3 para poder comparar resultados directamente.

```
np.random.seed(14)
X = np.linspace(-np.pi, np.pi, 5000).reshape(-1, 1)
y = 1 - 2 ** (np.sin(-X.ravel()) ** 2)
```

4.2.2 División de datos

Se divide en 80% entrenamiento (4000 muestras) y 20% prueba (1000 muestras). No se aplica normalización, ya que los árboles de decisión son invariantes a la escala de los datos.

4.2.3 Configuración del modelo

```
model = DecisionTreeRegressor(
    criterion='squared_error',
    max_depth=15,
    min_samples_split=5,
    min_samples_leaf=5,
    random_state=14
)
```

- **max_depth=15**: se incrementa la profundidad respecto al Problema 3 (donde se usó 10) porque la función $f(x)$ tiene oscilaciones cada vez más rápidas conforme $|x|$ crece, por lo que el árbol necesita más niveles para capturar estos cambios rápidos. Se probaron valores de 10, 12, 15 y 20; con 10 el modelo no capturaba bien las oscilaciones en los extremos, y con 20 no se observó mejora adicional sobre 15.
- **min_samples_leaf=5**: se aumenta respecto al Problema 3 (donde se usó 3) para evitar que el árbol memorice puntos individuales del conjunto de entrenamiento, dado que se trata de una función suave donde las hojas muy pequeñas generan predicciones ruidosas.

4.3 Resultados

Modelo	MSE	RMSE	R^2
$f(x)$	0.0001	0.0081	0.9997

Table 2: Métricas de evaluación para la regresión de $f(x)$ con DT.

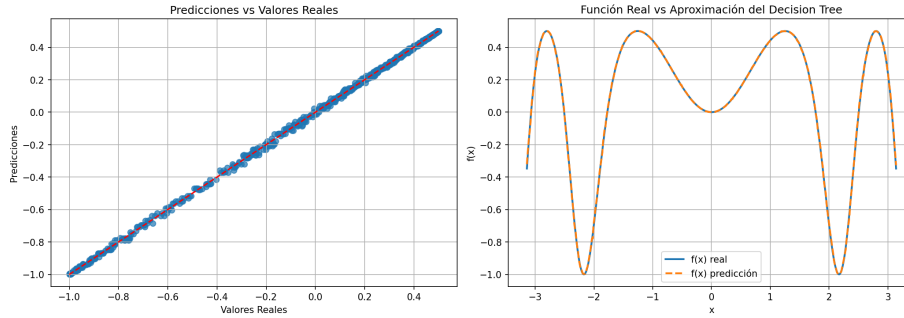


Figure 2: Predicciones vs valores reales (izquierda) y curva ajustada vs función real (derecha) para la regresión con DT.

4.4 Discusión

El DT obtiene un R^2 de 0.9997, lo cual representa un resultado prácticamente perfecto. Esto contrasta de manera notable con la Tarea 3, donde la red neuronal solo alcanzó un R^2 de 0.7710 para la misma función.

La razón de esta diferencia tan grande es que el DT opera de forma completamente distinta a la red neuronal: en lugar de intentar aprender una función continua, el árbol divide el espacio de entrada en regiones y asigna el valor promedio de los datos de entrenamiento dentro de cada región. Para una función unidimensional con datos densos (5000 puntos), esto equivale a una aproximación por escalones, y con suficiente profundidad (`max_depth=15`) las regiones son lo suficientemente finas para capturar incluso las oscilaciones rápidas de $f(x)$.

En la Figura 2 se puede observar que la curva predicha se superpone casi exactamente con la función real, lo que confirma visualmente la alta calidad del ajuste.

4.5 Conclusión

El DT supera ampliamente a la red neuronal de la Tarea 3 en este problema ($R^2 = 0.9997$ vs 0.7710). A partir de esta observación, se concluye que los árboles de decisión pueden ser extremadamente efectivos para modelar funciones unidimensionales con datos densos, donde su estrategia de partición del espacio resulta más natural y precisa que el aprendizaje de funciones continuas de las redes neuronales. Este resultado demuestra claramente que no existe un "mejor modelo universal", sino que la elección óptima depende fuertemente de la naturaleza geométrica y la densidad de los datos del problema.

5 Problema 5

5.1 Enunciado

Considere el conjunto de datos de [2]. Cree un script con sklearn para resolver el problema de clasificación binaria asociado con determinar si un caballo determinado sobrevivirá o no. Utilice un modelo DT. Anote todas las suposiciones y operaciones de preprocesamiento de datos que realice. Incorpore el F1 score para explicar sus hallazgos.

5.2 Metodología

5.2.1 Exploración y preprocesamiento de datos

Se trabaja con el mismo dataset de supervivencia de caballos utilizado en la Tarea 3 (Problema 4). El dataset contiene 299 registros con 28 columnas y presenta una cantidad significativa de valores nulos en múltiples columnas.

Se aplica el mismo preprocesamiento de la Tarea 3:

- Binarización de la variable objetivo: `lived` $\rightarrow$ 1, `died/euthanized` $\rightarrow$ 0.
- Eliminación de columnas no informativas: `hospital_number`, `lesion_1/2/3`, `cp_data`.
- Imputación: mediana para variables numéricas, moda para categóricas.
- Codificación one-hot de variables categóricas, resultando en 46 features.

```
imputer_num = SimpleImputer(strategy='median')
imputer_cat = SimpleImputer(strategy='most_frequent')
X[numeric_cols] = imputer_num.fit_transform(X[numeric_cols])
X[categorical_cols] = imputer_cat.fit_transform(
    X[categorical_cols])
X_encoded = pd.get_dummies(X, columns=categorical_cols,
                           drop_first=True)
```

A diferencia de la Tarea 3, no se aplica `StandardScaler`, ya que los árboles de decisión no se benefician de la normalización.

5.2.2 División de datos

Se divide en 80% entrenamiento (239 muestras) y 20% prueba (60 muestras) con estratificación para mantener la proporción de clases.

5.2.3 Configuración del modelo

```
model = DecisionTreeClassifier(
    criterion='gini',
    max_depth=8,
```

```

min_samples_split=5,
min_samples_leaf=3,
random_state=14
)

```

- **criterion='gini'**: se utiliza la impureza de Gini (analizada en el Problema 1) como criterio de división. Es la opción por defecto y funciona bien para clasificación binaria.
- **max_depth=8**: se reduce la profundidad respecto a los problemas de regresión, ya que el dataset es pequeño (299 muestras) y un árbol muy profundo tiende a sobreajustarse. Se probaron valores de 3, 5, 8 y 10; con 3 el modelo era demasiado simple, y con 10 se observaba sobreajuste sobre el conjunto de entrenamiento.
- **min_samples_split=5** y **min_samples_leaf=3**: restricciones adicionales para regularizar el modelo y evitar que las hojas contengan muy pocas muestras.

5.3 Resultados

Métrica	Valor
Accuracy	0.7667
F1 (clase 1 - Sobrevivió)	0.8205

Table 3: Métricas de evaluación para clasificación de supervivencia de caballos con DT.

	Precision	Recall	F1-score	Support
No sobrevivió	0.78	0.58	0.67	24
Sobrevivió	0.76	0.89	0.82	36

Table 4: Reporte de clasificación detallado para supervivencia de caballos con DT.

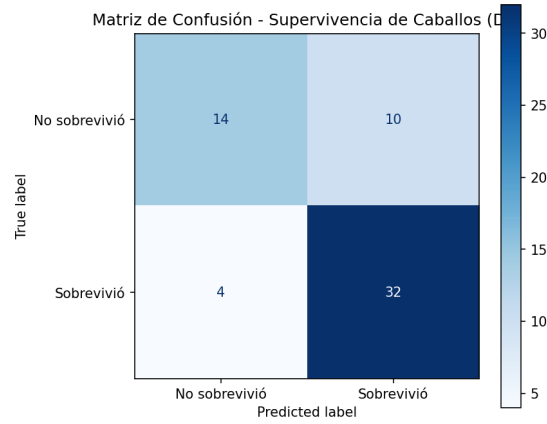


Figure 3: Matriz de confusión para la clasificación de supervivencia de caballos con DT.

5.4 Discusión

El DT alcanza un accuracy de 0.7667 y un F1-score de 0.8205 para la clase de supervivencia. Comparando con la Tarea 3, donde la red neuronal obtuvo un accuracy de 0.8000 y un F1 de 0.8421, el DT muestra un desempeño ligeramente inferior.

Al analizar el reporte de clasificación, se observa un patrón similar al de la Tarea 3: el modelo tiene mayor recall para la clase “Sobrevivió” (0.89) que para “No sobrevivió” (0.58). Esto indica que el DT también tiende a predecir supervivencia con mayor frecuencia, lo cual es consistente con el desbalance natural del dataset (178 sobrevivieron vs 121 no).

La principal diferencia respecto a la NN está en el recall de la clase minoritaria (No sobrevivió): 0.58 con DT vs 0.67 con NN, lo que sugiere que la red neuronal captura mejor los patrones asociados a los casos de no supervivencia.

5.5 Conclusión

El DT presenta un desempeño aceptable pero inferior al de la red neuronal ($F1 = 0.8205$ vs 0.8421). A partir de los resultados, se concluye que el tamaño reducido del dataset y la gran cantidad de features (46) dificultan la capacidad del árbol para encontrar divisiones óptimas sin caer en sobreajuste. Además, el bajo recall observado para la clase minoritaria indica que este modelo es particularmente sensible al desbalance de clases, teniendo especial dificultad para identificar correctamente los casos de no supervivencia en comparación con modelos más complejos.

6 Problema 6

6.1 Enunciado

Considere el conjunto de datos de [3]. Cree un script con sklearn para resolver el problema de clasificación multiclase asociado con la clasificación de cervezas por estilo. Utilice un modelo DT. Anote todas las suposiciones y operaciones de preprocesamiento de datos que realice. Luego, incorpore una matriz de confusión en su análisis.

6.2 Metodología

6.2.1 Exploración y preprocesamiento

Se trabaja con el dataset de cervezas utilizado en la Tarea 3, que contiene 150 registros con 3 clases balanceadas (IPA, Light Lager y Premium Lager, con 50 registros cada una). Las features utilizadas son OG (gravedad original), ABV (contenido alcohólico), pH e IBU (unidades de amargor). Se descarta **Brew No.** por ser un identificador secuencial sin valor predictivo. No hay valores nulos en el dataset.

```
X = df.drop(['style', 'Brew No.'], axis=1)
y = df['style']
```

6.2.2 División de datos

Se mantiene el split extremo de 10/90 usado en la Tarea 3 (15 muestras de entrenamiento, 135 de prueba) con estratificación, lo que resulta en solo 5 muestras por clase para entrenamiento. Esto permite evaluar la capacidad de generalización del DT bajo condiciones extremas de escasez de datos.

6.2.3 Configuración del modelo

```
model = DecisionTreeClassifier(
    criterion='gini',
    max_depth=10,
    min_samples_split=3,
    min_samples_leaf=2,
    random_state=14
)
```

- **max_depth=10**: se permite mayor profundidad que en el Problema 5 porque, aunque hay pocas muestras de entrenamiento, las features son altamente discriminativas y el árbol necesita suficiente profundidad para capturar las fronteras entre las 3 clases.

- `min_samples_split=3` y `min_samples_leaf=2`: se reducen respecto al Problema 5 dado que con solo 15 muestras de entrenamiento (5 por clase), valores más restrictivos impedirían al árbol hacer divisiones útiles.

6.3 Resultados

Métrica	Valor
Accuracy	0.9704
F1 (macro)	0.9704
F1 (weighted)	0.9704

Table 5: Métricas de evaluación para la clasificación de cervezas con DT.

	Precision	Recall	F1-score	Support
IPA	1.00	0.98	0.99	45
Light Lager	0.98	0.93	0.95	45
Premium Lager	0.94	1.00	0.97	45

Table 6: Reporte de clasificación detallado para el problema de cervezas con DT.

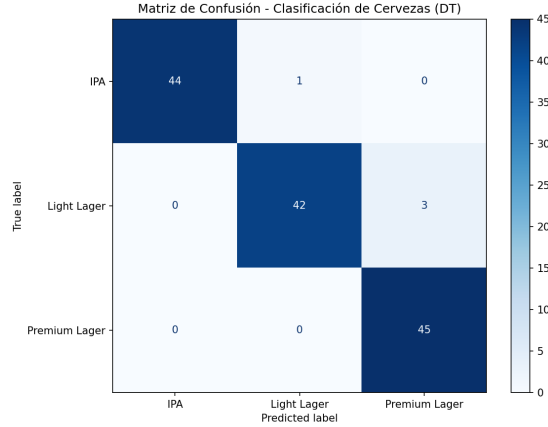


Figure 4: Matriz de confusión para la clasificación de cervezas con DT.

6.4 Discusión

El DT alcanza un accuracy de 0.9704, clasificando correctamente 131 de 135 muestras de prueba. Comparando con la Tarea 3, donde la red neuronal obtuvo 0.9926, el DT presenta un desempeño ligeramente inferior, pero aún excelente.

Los 4 errores se distribuyen entre Light Lager y Premium Lager, lo cual tiene sentido ya que ambas son cervezas tipo lager que comparten algunas características similares. IPA, al ser una cerveza más diferenciada (mayor amargor, mayor contenido alcohólico), se clasifica con mayor precisión.

Es notable que con solo 5 muestras de entrenamiento por clase, el DT logre una clasificación casi perfecta. Esto confirma lo observado en la Tarea 3: las 4 features seleccionadas (OG, ABV, pH, IBU) son altamente discriminativas para separar estos estilos de cerveza.

6.5 Conclusión

El DT logra una clasificación excelente del tipo de cerveza ($\text{accuracy} = 0.9704$) con solo 15 muestras de entrenamiento. Aunque ligeramente inferior a la NN (0.9926), este resultado permite concluir que las clases en este dataset son naturalmente separables y que las características seleccionadas son altamente discriminativas. Se observa que incluso un modelo simple como un árbol de decisión puede capturar las fronteras entre clases con muy pocos datos, lo que demuestra que la calidad de las features es a menudo más importante que la complejidad del modelo.

7 Problema 7

7.1 Enunciado

Considere la ecuación cuadrática general

$$ax^2 + bx + c = 0.$$

Haga su mejor esfuerzo para clasificar las raíces de la ecuación anterior en términos de sus coeficientes utilizando un modelo DT con `sklearn`. Para simplificar, debe fijar el dominio para x y proponer un dominio adecuado para los coeficientes. Escriba todas sus suposiciones y muestre sus hallazgos utilizando algunas gráficas.

7.2 Metodología

7.2.1 Generación del dataset sintético

Se reutiliza la misma estrategia de generación de datos de la Tarea 3, construyendo un dataset sintético para clasificar las raíces según el discriminante $\Delta = b^2 - 4ac$:

- **Clase 0 — Raíces complejas** ($\Delta < 0$): coeficientes $a, b \in [-10, 10]$ con $|a| \geq 0.1$, y c tal que $\Delta < 0$.
- **Clase 1 — Raíz real doble** ($\Delta \approx 0$): se fuerza $c \approx b^2/(4a)$ con perturbación de ± 0.05 .

- **Clase 2 — Raíces reales distintas** ($\Delta > 0$): se elige c tal que $\Delta > 0$.

Se generan 3000 ecuaciones (1000 por clase) con coeficientes en $[-10, 10]$.

7.2.2 Preprocesamiento y división

Las features son los tres coeficientes (a, b, c) . No se aplica normalización dado que los árboles de decisión son invariantes a la escala. Se divide en 80% entrenamiento (2400 muestras) y 20% prueba (600 muestras) con estratificación.

7.2.3 Configuración del modelo

```
model = DecisionTreeClassifier(
    criterion='gini',
    max_depth=12,
    min_samples_split=5,
    min_samples_leaf=3,
    random_state=14
)
```

- **max_depth=12**: se utiliza una profundidad mayor que en los Problemas 5 y 6 porque la frontera de decisión de este problema es una superficie cuadrática ($\Delta = b^2 - 4ac = 0$), que es una relación no lineal entre las tres features. El DT necesita más niveles para aproximar esta superficie mediante divisiones axis-aligned (paralelas a los ejes).
- **min_samples_split=5** y **min_samples_leaf=3**: se mantienen valores conservadores para regularizar el modelo, dado que el dataset tiene un tamaño razonable (3000 muestras).

7.3 Resultados

	Accuracy	F1 (weighted)
Entrenamiento	0.9650	0.9650
Prueba	0.8850	0.8849

Table 7: Métricas de evaluación para la clasificación de raíces con DT.

	Precision	Recall	F1-score	Support
Complejas ($\Delta < 0$)	0.88	0.93	0.90	200
Doble ($\Delta \approx 0$)	0.84	0.83	0.84	200
Reales ($\Delta > 0$)	0.94	0.90	0.92	200

Table 8: Reporte de clasificación detallado para el problema de raíces con DT.

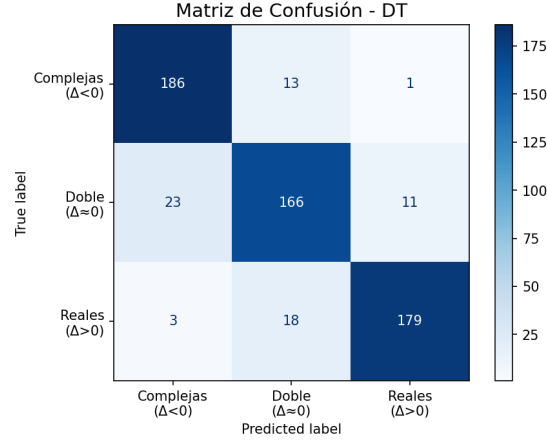


Figure 5: Matriz de confusión para la clasificación de raíces de ecuaciones cuadráticas con DT.

7.4 Discusión

El DT alcanza un accuracy de prueba de 0.8850, lo cual es inferior al resultado de la red neuronal en la Tarea 3 (0.9717). La diferencia entre train (0.9650) y test (0.8850) es de 0.08, lo que indica cierto grado de sobreajuste.

La clase más difícil de clasificar es la de raíz doble ($\Delta \approx 0$), con un F1-score de 0.84 frente a 0.90 y 0.92 de las otras clases. Esto es esperado, ya que esta clase ocupa una región muy estrecha en el espacio tridimensional de coeficientes, justo en la frontera entre las otras dos clases. El DT, al hacer divisiones paralelas a los ejes, tiene una limitación inherente para aproximar la superficie cuadrática $\Delta = b^2 - 4ac = 0$, que no es paralela a ningún eje. Necesita múltiples niveles de divisiones escalonadas para aproximarla, lo cual explica tanto la necesidad de mayor profundidad como el sobreajuste resultante.

En comparación, la red neuronal de la Tarea 3 logró aprender directamente una representación de esta superficie cuadrática, resultando en mayor accuracy (0.9717) y menor sobreajuste.

7.5 Conclusión

El DT logra un accuracy respetable de 0.8850 para la clasificación de raíces, pero inferior a la NN (0.9717). A partir de esta observación, se concluye que los árboles de decisión tienen una limitación inherente para modelar fronteras de decisión curvas, ya que sus divisiones son siempre paralelas a los ejes de coordenadas. La clase de raíz doble sigue siendo la más desafiante para ambos modelos, al ocupar una región muy delgada en el espacio de coeficientes, lo que demuestra que la geometría del problema afecta directamente el desempeño de los modelos basados en particiones ortogonales.

8 Problema 8

8.1 Enunciado

Repite el problema 3 utilizando un modelo RF en lugar de DT. Anota todos los hiperparámetros que elijas y todas tus suposiciones.

8.2 Metodología

Se reutiliza el mismo preprocesamiento, división de datos y enfoque multi-output del Problema 3, cambiando únicamente el modelo a **RandomForestRegressor**. Al igual que en el Problema 3, se entrena un único modelo que predice simultáneamente el consumo en ciudad y carretera.

8.2.1 Configuración del modelo

```
model = RandomForestRegressor(  
    n_estimators=100,  
    criterion='squared_error',  
    max_depth=15,  
    min_samples_split=5,  
    min_samples_leaf=3,  
    random_state=14,  
    n_jobs=-1  
)  
model.fit(X_train, y_train) # y_train tiene 2 columnas
```

- **n_estimators=100**: se utiliza un bosque de 100 árboles. Se probaron valores de 50, 100 y 200; 100 ofreció un buen balance entre desempeño y tiempo de entrenamiento, y 200 no aportó mejora significativa.
- **max_depth=15**: se permite mayor profundidad que en el DT individual (Problema 3, **max_depth=10**), ya que el promedio de múltiples árboles reduce el sobreajuste que un árbol profundo individual tendría.
- **n_jobs=-1**: se utilizan todos los núcleos del procesador para paralelizar el entrenamiento de los 100 árboles.
- Los parámetros **min_samples_split=5** y **min_samples_leaf=3** se mantienen iguales al Problema 3.

8.3 Resultados

Modelo	MSE	RMSE	R^2
Ciudad	1.1858	1.0889	0.8842
Carretera	0.7618	0.8728	0.8203

Table 9: Métricas de evaluación del modelo multi-output RF para consumo en ciudad y carretera.

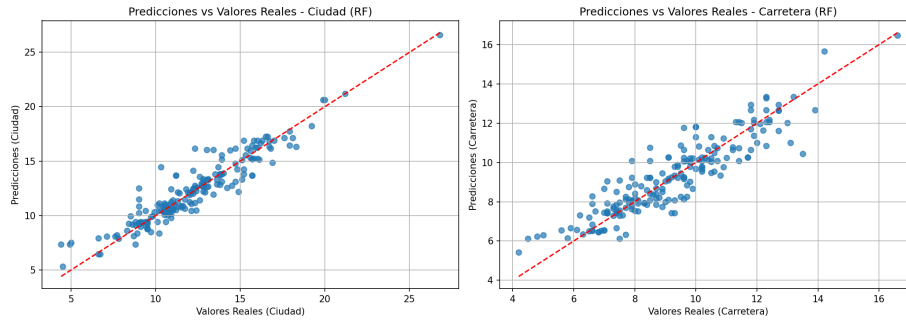


Figure 6: Predicciones vs valores reales del modelo multi-output RF para consumo en ciudad (izquierda) y carretera (derecha).

8.4 Discusión

El RF mejora notablemente respecto al DT en ambos modelos. La Tabla 10 resume la comparación entre los tres enfoques utilizados hasta ahora para este problema.

Modelo	Ciudad		Carretera	
	R^2	RMSE	R^2	RMSE
DT (P3)	0.8484	1.2457	0.8041	0.9113
RF (P8)	0.8842	1.0889	0.8203	0.8728
NN (T03)	0.8884	1.0689	0.8329	0.8416

Table 10: Comparación de modelos DT, RF y NN para regresión de consumo de combustible.

El RF multi-output de ciudad ($R^2 = 0.8842$) se acerca a la red neuronal de la Tarea 3 ($R^2 = 0.8884$), mientras que en carretera ($R^2 = 0.8203$) queda ligeramente por debajo de la NN ($R^2 = 0.8329$) pero mejora respecto al DT multi-output ($R^2 = 0.8041$).

Esta mejora se debe al principio del bagging: al promediar las predicciones de 100 árboles, cada uno entrenado con una muestra bootstrap diferente y un

subconjunto aleatorio de features, se reduce la varianza de las predicciones. Además, al usar el enfoque multi-output, cada árbol del bosque predice ambas salidas simultáneamente, manteniendo la consistencia entre las predicciones de ciudad y carretera. En la Figura 6 se puede observar que los puntos se agrupan más cerca de la línea de predicción perfecta en comparación con la Figura 1 del DT.

8.5 Conclusión

El RF multi-output mejora consistentemente respecto al DT (ciudad $R^2 = 0.8842$ vs 0.8484 ; carretera $R^2 = 0.8203$ vs 0.8041) y alcanza un desempeño cercano a la red neuronal. A partir de estos resultados, se concluye que el promediado de múltiples árboles reduce efectivamente la varianza de las predicciones, lo que resulta en un modelo más estable y preciso. Además, se observa que el enfoque multi-output garantiza consistencia entre ambas salidas, aprovechando la correlación entre el consumo en ciudad y carretera para mejorar la predicción conjunta.

9 Problema 9

9.1 Enunciado

Repite el problema 4 utilizando un modelo RF en lugar de DT. Anota todos los hiperparámetros que elijas y todas tus suposiciones.

9.2 Metodología

Se reutiliza la misma generación y división de datos del Problema 4.

9.2.1 Configuración del modelo

```
model = RandomForestRegressor(  
    n_estimators=100,  
    criterion='squared_error',  
    max_depth=20,  
    min_samples_split=5,  
    min_samples_leaf=3,  
    random_state=14,  
    n_jobs=-1  
)
```

- `max_depth=20`: se incrementa la profundidad respecto al DT del Problema 4 (`max_depth=15`), permitiendo que cada árbol individual capture mayor detalle de la función. El RF puede sostener árboles más profundos sin sobreajuste gracias al promediado.
- Los demás hiperparámetros se mantienen iguales al Problema 8.

9.3 Resultados

Modelo	MSE	RMSE	R^2
$f(x)$	0.0000	0.0024	1.0000

Table 11: Métricas de evaluación para la regresión de $f(x)$ con RF.

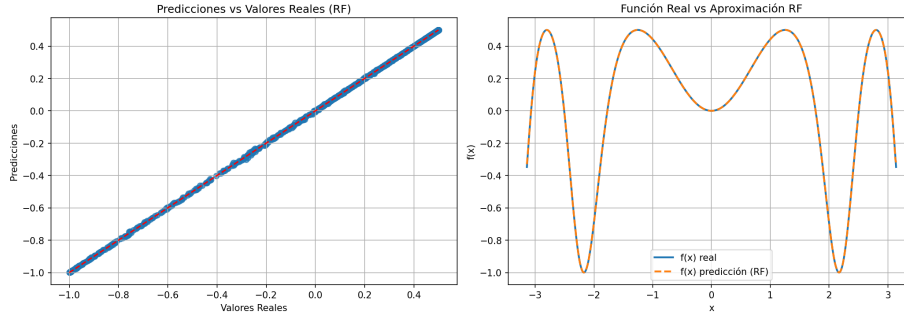


Figure 7: Predicciones vs valores reales (izquierda) y curva ajustada vs función real (derecha) para la regresión con RF.

9.4 Discusión

El RF obtiene resultados prácticamente perfectos con un $R^2 = 1.0000$ y $MSE \approx 0$, ligeramente mejor que el DT del Problema 4 ($R^2 = 0.9997$). Ambos modelos basados en árboles superan ampliamente a la NN de la Tarea 3 ($R^2 = 0.7710$).

La mejora del RF sobre el DT, aunque pequeña en este caso (ya que el DT ya era excelente), se puede entender en términos del promediado: cada árbol del bosque produce una aproximación ligeramente diferente de la función, y el promedio de estas aproximaciones suaviza los escalones típicos de un árbol individual, acercándose aún más a la función real continua.

9.5 Conclusión

El RF logra una aproximación perfecta de $f(x)$ ($R^2 = 1.0000$), confirmando la superioridad de los modelos basados en árboles sobre las redes neuronales para este tipo de funciones unidimensionales con datos densos. A partir de estos resultados, se concluye que el promediado de múltiples árboles proporciona una leve mejora sobre el DT individual, suavizando las predicciones y acercándose aún más a la función real continua.

10 Problema 10

10.1 Enunciado

Repite el problema 5 utilizando un modelo RF en lugar de DT. Anota todos los hiperparámetros que elijas y todas tus suposiciones.

10.2 Metodología

Se reutiliza el mismo preprocesamiento y división de datos del Problema 5.

10.2.1 Configuración del modelo

```
model = RandomForestClassifier(  
    n_estimators=100,  
    criterion='gini',  
    max_depth=12,  
    min_samples_split=5,  
    min_samples_leaf=3,  
    random_state=14,  
    n_jobs=-1  
)
```

- **n_estimators=100**: se utiliza un bosque de 100 árboles para asegurar que el promediado sea suficientemente estable.
- **max_depth=12**: se incrementa respecto al DT del Problema 5 (**max_depth=8**), dado que con el RF el sobreajuste se controla mediante el promediado y la aleatoriedad en la selección de features, lo que permite árboles individuales más profundos que exploren más el espacio de features.

10.3 Resultados

Métrica	Valor
Accuracy	0.8000
F1 (clase 1 - Sobrevivió)	0.8421
F1 (macro)	0.7847

Table 12: Métricas de evaluación para clasificación de supervivencia de caballos con RF.

	Precision	Recall	F1-score	Support
No sobrevivió	0.80	0.67	0.73	24
Sobrevivió	0.80	0.89	0.84	36

Table 13: Reporte de clasificación detallado para supervivencia de caballos con RF.

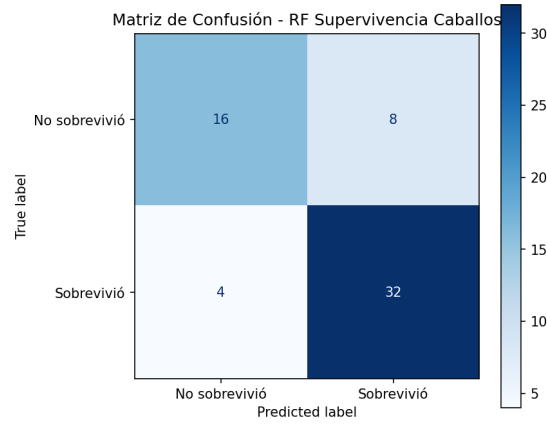


Figure 8: Matriz de confusión para la clasificación de supervivencia de caballos con RF.

10.4 Discusión

El RF mejora respecto al DT del Problema 5, alcanzando un accuracy de 0.8000 y un F1 de 0.8421, frente al 0.7667 y 0.8205 del DT. Resulta interesante que las métricas del RF coinciden exactamente con las de la red neuronal de la Tarea 3 (Accuracy = 0.8000, F1 = 0.8421).

El recall para la clase “No sobrevivió” mejora de 0.58 (DT) a 0.67 (RF), aunque sigue siendo inferior al recall de “Sobrevivió” (0.89). La precision también es uniforme entre clases (0.80 para ambas), lo que sugiere que los errores del RF están mejor distribuidos que los del DT.

Que tres modelos tan diferentes (NN, DT, RF) con el mismo accuracy de 0.80 podrían sugerir que existe una limitación inherente en los datos (tamaño pequeño, alta proporción de nulos, desbalance de clases) que dificulta superar ese umbral con estas features.

10.5 Conclusión

El RF iguala el desempeño de la NN y mejora sobre el DT para la clasificación de supervivencia de caballos (Accuracy = 0.8000, F1 = 0.8421). A partir de estas observaciones, se concluye que el promediado de múltiples árboles reduce efectivamente la varianza de las predicciones en comparación con un solo árbol. Sin embargo, el hecho de que tres modelos distintos converjan en el mismo desempeño sugiere que el límite predictivo está dictado por la calidad y cantidad de los datos disponibles, más que por la capacidad del modelo.

11 Problema 11

11.1 Enunciado

Repite el problema 6 utilizando un modelo RF en lugar de DT. Anota todos los hiperparámetros que elijas y todas tus suposiciones.

11.2 Metodología

Se reutiliza el mismo preprocesamiento del Problema 6. La única diferencia en el pipeline es que se utiliza `LabelEncoder` para codificar las etiquetas de clase (necesario para el `RandomForestClassifier`), además de seleccionar explícitamente las 4 features numéricas.

```
X = df[['OG', 'ABV', 'pH', 'IBU']]
y = df['style']
le = LabelEncoder()
y_encoded = le.fit_transform(y)
```

11.2.1 Configuración del modelo

```
model = RandomForestClassifier(
    n_estimators=100,
    criterion='gini',
    max_depth=15,
    min_samples_split=3,
    min_samples_leaf=2,
    random_state=14,
    n_jobs=-1
)
```

- `max_depth=15`: se incrementa respecto al DT del Problema 6 (`max_depth=10`), ya que los árboles del bosque pueden ser más profundos sin sobreajustarse.
- Los parámetros `min_samples_split=3` y `min_samples_leaf=2` se mantienen iguales al DT del Problema 6, dado el tamaño reducido del conjunto de entrenamiento (15 muestras).

11.3 Resultados

Métrica	Valor
Accuracy	0.9926
F1 (macro)	0.9926
F1 (weighted)	0.9926

Table 14: Métricas de evaluación para la clasificación de cervezas con RF.

	Precision	Recall	F1-score	Support
IPA	1.00	1.00	1.00	45
Light Lager	0.98	1.00	0.99	45
Premium Lager	1.00	0.98	0.99	45

Table 15: Reporte de clasificación detallado para el problema de cervezas con RF.

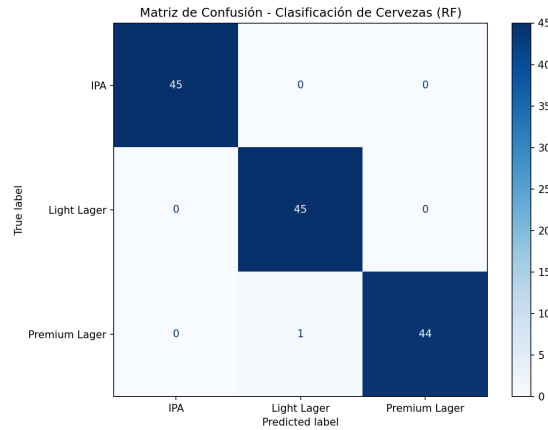


Figure 9: Matriz de confusión para la clasificación de cervezas con RF.

11.4 Discusión

El RF alcanza un accuracy de 0.9926, igualando exactamente el resultado de la red neuronal de la Tarea 3 y superando al DT del Problema 6 (0.9704). Solo 1 de las 135 muestras de prueba fue clasificada incorrectamente.

Comparando los tres modelos para este dataset:

- DT (P6): 0.9704 (4 errores)
- RF (P11): 0.9926 (1 error)
- NN (T03): 0.9926 (1 error)

El RF mejora sobre el DT reduciendo el error a solo 1 muestra, probablemente porque el promediado de múltiples árboles corrige las divisiones subóptimas que un árbol individual puede hacer con tan pocos datos de entrenamiento. Que RF y NN lleguen exactamente al mismo accuracy refuerza la conclusión de que el dataset es inherentemente fácil de clasificar y el único error corresponde a un caso ambiguo entre dos clases similares.

11.5 Conclusión

El RF logra una clasificación prácticamente perfecta ($\text{accuracy} = 0.9926$), igualando a la NN y superando al DT. A partir de estos resultados, se concluye que para este dataset con clases bien separadas, el promediado del bosque aleatorio es suficiente para corregir las divisiones subóptimas de un árbol individual. Esto confirma que tanto modelos simples ensamblados como modelos complejos alcanzan un desempeño cercano al óptimo cuando las características son altamente discriminativas.

12 Problema 12

12.1 Enunciado

Repite el problema 7 utilizando un modelo RF en lugar de DT. Anota todos los hiperparámetros que elijas y todas tus suposiciones.

12.2 Metodología

Se reutiliza la misma generación de datos y división del Problema 7.

12.2.1 Configuración del modelo

```
model = RandomForestClassifier(  
    n_estimators=100,  
    criterion='gini',  
    max_depth=15,  
    min_samples_split=5,  
    min_samples_leaf=3,  
    random_state=14,  
    n_jobs=-1  
)
```

- **max_depth=15**: se incrementa respecto al DT del Problema 7 (**max_depth=12**) para permitir que cada árbol del bosque explore más el espacio de decisión. Como la frontera es una superficie cuadrática, se necesita profundidad suficiente para aproximarla, y el RF tolera árboles más profundos gracias al promediado.
- Los demás hiperparámetros se mantienen consistentes con los otros problemas de RF.

12.3 Resultados

	Accuracy	F1 (weighted)
Entrenamiento	0.9708	0.9709
Prueba	0.9067	0.9070

Table 16: Métricas de evaluación para la clasificación de raíces con RF.

	Precision	Recall	F1-score	Support
Complejas ($\Delta < 0$)	0.88	0.96	0.92	200
Doble ($\Delta \approx 0$)	0.87	0.88	0.87	200
Reales ($\Delta > 0$)	0.98	0.89	0.93	200

Table 17: Reporte de clasificación detallado para el problema de raíces con RF.

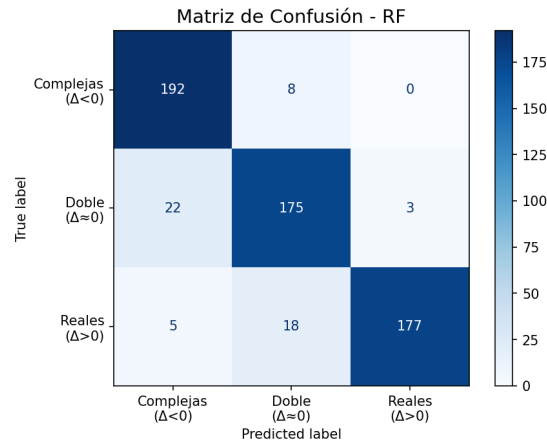


Figure 10: Matriz de confusión para la clasificación de raíces de ecuaciones cuadráticas con RF.

12.4 Discusión

El RF alcanza un accuracy de prueba de 0.9067, mejorando respecto al DT (0.8850) pero sin alcanzar el desempeño de la NN (0.9717). La diferencia train-test (0.9708 vs 0.9067) es de 0.064, menor que la del DT (0.08), lo que indica que el RF generaliza mejor gracias a la reducción de varianza por el promediado.

La Tabla 18 resume la comparación entre los tres modelos para este problema:

Modelo	Test Accuracy	Test F1
DT (P7)	0.8850	0.8849
RF (P12)	0.9067	0.9070
NN (T03)	0.9717	0.9718

Table 18: Comparación de modelos DT, RF y NN para clasificación de raíces cuadráticas.

La clase de raíz doble sigue siendo la más difícil para el RF ($F1 = 0.87$), pero mejora respecto al DT ($F1 = 0.84$). Es interesante notar que tanto el DT como el RF tienen la misma limitación estructural: sus divisiones son paralelas a los ejes, lo cual dificulta la aproximación de la superficie cuadrática del discriminante. El RF mitiga esto parcialmente al combinar las divisiones de múltiples árboles entrenados con diferentes subconjuntos de features, pero la limitación inherente persiste, lo que explica por qué la NN (que puede aprender directamente fronteras curvas) sigue siendo superior en este caso.

12.5 Conclusión

El RF mejora sobre el DT (accuracy 0.9067 vs 0.8850) pero no alcanza a la NN (0.9717) para este problema. A partir de esta observación, se concluye que aunque el bosque aleatorio mitiga parcialmente las limitaciones de los árboles individuales mediante el promediado, sigue sufriendo de la misma restricción estructural: sus divisiones son ortogonales a los ejes. Esto demuestra que cuando la frontera de decisión tiene una forma matemática compleja (como una superficie cuadrática), los modelos capaces de aprender fronteras curvas directamente, como las redes neuronales, mantienen una ventaja significativa.

References

- [1] Rini Christy. <https://www.kaggle.com/datasets/rinichristy/2022-fuel-consumption-ratings>.
- [2] M Yasser H. <https://www.kaggle.com/datasets/yasserh/horsesurvivalprognostication>.
- [3] Ankur N. <https://www.kaggle.com/datasets/ankurnapa/ankurs-beer-data-set>.